1- SQL injection na busca:
curl -i -G --data-urlencode "nome=' OR 1=1 --" http://localhost:8080/clientes
curl -i -G --data-urlencode "nome=x'; DROP TABLE clientes; --" http://localhost:8080/clientes
curl -i "http://localhost:8080/clientes/%27%20OR%201%3D1%20--"

#	Requisição	Resposta	o que aconteceu
1	busca com ' OR 1=1 --	200, totalElementos: 0	O texto foi tratado como um nome qualquer. Nenhum cliente tem esse nome, então voltou vazio
2	busca com x'; DROP TABLE clientes; --	200, totalElementos: 0	Mesmo comportamento. O DROP TABLE não foi executado
3	GET /clientes/' OR 1=1 -- (na URL)	404	O texto foi interpretado como um código de cliente, e esse código não existe

Os 3 casos passaram no teste.

2. % e _ na busca:
for n in "%" "_" "Ana_Silva"; do echo "== nome=$n"; curl -s -o /dev/null -w "HTTP %{http_code}\n" -G --data-urlencode "nome=$n" http://localhost:8080/clientes; curl -s -G --data-urlencode "nome=$n" http://localhost:8080/clientes | grep -o '"totalElementos":[0-9]*'; done

#	Busca	Resposta	Esperado	Significado	Veredito
1	nome=%	HTTP 200, totalElementos: 53	0 resultados	O % casou com qualquer texto e devolveu os 53 clientes ativos, ou seja, todos	Falhou
2	nome=_	HTTP 200, totalElementos: 53	0 resultados	O _ casou com “um caractere qualquer”, e todo nome tem pelo menos um	Falhou
3	nome=Ana_Silva	HTTP 200, totalElementos: 1	0 resultados	O _ casou com o espaço de “Ana Silva”, encontrando um nome que não é o digitado	Falhou

Os 3 casos falharam no teste

3- JSON mal formado com requisições estranhas:
curl -i -X POST http://localhost:8080/clientes -H "Content-Type: application/json" -d '{"nome":"Zz QA", "email":'
curl -i -X POST http://localhost:8080/clientes -H "Content-Type: application/json" -d '{"nome":["a","b"]}'
curl -i -X POST http://localhost:8080/clientes -H "Content-Type: application/json" -d "
curl -i -X POST http://localhost:8080/clientes -d 'nome=Zz'
curl -i -X DELETE http://localhost:8080/clientes/CLI-0007
curl -i "http://localhost:8080/clientes?pagina=abc"

#	Requisição	Resposta	Esperado	Significado	Veredito
1	JSON cortado: `{"nome":"Zz QA", "email":`	`400`	`400`	O back detectou o JSON quebrado e recusou sem expor o motivo	**Passou**
2	Tipo errado: `{"nome":["a","b"]}`	`400`	`400`	Nome veio como lista em vez de texto, e foi recusado	**Passou**
3	Corpo vazio (`-d ''`)	`400`	`400`	Sem corpo não há o que cadastrar	**Passou**
4	Sem `Content-Type` (`-d 'nome=Zz'`)	`415`	`415`	O back só aceita JSON e disse isso pelo código certo	**Passou**
5	DELETE /clientes/CLI-0007	`405`	`405`	Não existe exclusão na API, então a regra “nunca excluir” vale também fora da tela. Nenhum dado foi apagado	**Passou**
6	GET /clientes?pagina=abc	`400`	`400`	Valor que não é número foi recusado	**Passou**

Os 6 casos passaram no teste.

4- Campos gigantes

```
BIG=$(head -c 5000 /dev/zero | tr '\0' 'A')
curl -i -X POST http://localhost:8080/clientes -H "Content-Type: application/json" \
 -d "{\"nome\":\"$BIG\",\"email\":\"qa.sec03@exemplo.com\",\"telefone\":\"(81)
91111-1111\",\"cidade\":\"Recife\"}"
curl -i -G --data-urlencode "nome=$BIG" http://localhost:8080/clientes

{ printf '{"nome":"'; head -c 200000 /dev/zero | tr '\0' 'A'; printf
'","email":"qa.sec04@exemplo.com","telefone":"(81) 91111-1111","cidade":"Recife"}'; } >
~/grande.json
curl -i -X POST http://localhost:8080/clientes -H "Content-Type: application/json"
--data-binary @$HOME/grande.json
```

#	Requisição	Resposta	Esperado	Significado	Veredito
1	Cadastro com nome de 5.000 letras	`400`, `{"erro":"nome deve ter no máximo 120 caracteres"}`	`400`	O limite do nome funcionou e nada foi gravado	**Passou**
2	Busca com 5.000 letras em `nome`	`200`, 0 resultados	200 ou 400, sem erro interno	O texto enorme foi tratado como um nome qualquer, sem travar	**Passou**
3	Cadastro com corpo de ~200 KB (nome de 200.000 letras)	`400`	`400`	O back leu o corpo grande, validou e recusou com a mesma mensagem, sem derrubar o servidor	**Passou**

Os 3 casos passaram no teste.

5- parar o banco

> etapa 1: banco ligado

```
curl -s -m 30 -w "\nHTTP %{http_code} em %{time_total}s\n" http://localhost:8080/health
curl -s -m 30 -w "\nHTTP %{http_code} em %{time_total}s\n"
"http://localhost:8080/clientes?nome=silva" | tail -c 200
```

Resposta: 200 nos dois (back responde normalmente com banco ligado)

> etapa 2: banco parado:

```
curl -i -m 60 -w "\nHTTP %{http_code} em %{time_total}s\n" http://localhost:8080/health
curl -i -m 60 -w "\nHTTP %{http_code} em %{time_total}s\n"
"http://localhost:8080/clientes?nome=silva"
curl -i -m 60 -w "\nHTTP %{http_code} em %{time_total}s\n"
http://localhost:8080/clientes/CLI-0007
curl -i -m 60 -w "\nHTTP %{http_code} em %{time_total}s\n" -X POST
http://localhost:8080/clientes -H "Content-Type: application/json" \
 -d '{"nome":"Zz QA Banco Parado","email":"qa.parado01@exemplo.com","telefone":"(81)
91111-1111","cidade":"Recife"}'
```

Resposta: 503, falha na conexão com o banco de dados